How the program works

The program enables file upload through the JUCE fileChooser class reference declared in DeckGUI.h. By clicking the load button on the GUI, a file browser dialog appears and allows users to select an audio file. launchAsync is used to prevent UI from freezing. Users can also drag and drop audio files onto the deck to load files. This is enabled by the isInterestedInFileDrag and filesDropped functions.

```
268  void DeckGUI::buttonClicked(juce::Button *button)
306      /* event listener for load button */
307      if (button == &loadButton)
308      {
309          auto fileChooserFlags = juce::FileBrowserComponent::canSelectFiles;
310          fChooser.launchAsync(fileChooserFlags, [this](const juce::FileChooser &chooser)
311                               {
312              player->loadURL(juce::URL{chooser.getResult()});
313              waveformDisplay.loadURL(juce::URL{chooser.getResult()});
314
315              /* resets play state when new file is uploaded */
316              isPlaying = false;
317              player->stop();
318              playButton.setButtonText("PLAY");
319
320              /* set load state */
321              isLoaded = true;
322              std::cout << "file loaded" << std::endl; });
323      }
324  }
```

```
398  bool DeckGUI::isInterestedInFileDrag(const juce::StringArray &files)
399  {
400      std::cout << "DeckGUI::isInterestedInFileDrag" << std::endl;
401      return true;
402  }
403
404  void DeckGUI::filesDropped(const juce::StringArray &files, int x, int y)
405  {
406      std::cout << "DeckGUI::filesDropped" << std::endl;
407      if (files.size() == 1)
408      {
409          // set state to true to enable playButton press
410          isLoaded = true;
411          player->loadURL(juce::URL{juce::File{files[0]}});
412          waveformDisplay.loadURL(juce::URL{juce::File{files[0]}}); // Load into waveform
                 display as well
413      }
414  }
```

To play the file, users click the play button. If a file is not loaded, an error message would be triggered (lines 274-279 in DeckGUI.cpp). If a file has been loaded, clicking the play button will change the play state and the deck will start playing the audio file. The button text will also be changed to ‘STOP’. Clicking the button again will stop the playback and change the button text to be ‘PLAY’.

```
/* button event listener */
void DeckGUI::buttonClicked(juce::Button *button)
{
    /* event listener for play button */
    if (button == &playButton)
    {
        /* check if file is loaded */
        if (isLoaded == false)
        {
            DBG("no file loaded");
            juce::AlertWindow::showMessageBoxAsync(juce::MessageBoxIconType::WarningIcon, "No file loaded", "Please
                load audio file");
            return; // prevent further execution if no file is loaded
        }

        if (isLoaded)
        {
            /* changes play state */
            isPlaying = !isPlaying;

            /* if song is not playing*/
            if (isPlaying)
            {
                std::cout << "Playing song" << std::endl;
                player->start();
                playButton.setButtonText("STOP");
            }

            /* if song is playing */
            else if (!isPlaying)
            {
                std::cout << "Stopping song" << std::endl;
                player->stop();
                playButton.setButtonText("PLAY");
            }

            playButton.setToggleState(isPlaying, juce::NotificationType::dontSendNotification);
        }
    }
```

During audio playback, the centermost, largest disc which is the position slider, updates its slider position. Additionally, users can hover over the position slider and use the scroll wheel to adjust the slider position.

```
void DeckGUI::timerCallback()
{
    waveformDisplay.setPositionRelative(player->getPositionRelative());

    /* updates slider position during playback */
    if (isPlaying)
    {
        posSlider.setValue(player->getPositionRelative());
    }
}
```

Next to the position slider are the volume and speed sliders, as well as the equalizer dials. Below are dials that change audio reverb parameters, manipulating the audio space. To be able to adjust volume and speed levels, the AudioTransportSource and ResamplingAudioSource class are called and used in DJAudioPlayer::prepareToPlay().

```
private:
    juce::AudioFormatManager &formatManager;
    std::unique_ptr<juce::AudioFormatReaderSource> readerSource;
    juce::AudioTransportSource transportSource;
    juce::ResamplingAudioSource resampleSource{&transportSource, false, 2};

    juce::IIRFilterAudioSource low{&resampleSource, false};
    juce::IIRFilterAudioSource mid{&low, false};
    juce::IIRFilterAudioSource high{&mid, false};

    juce::ReverbAudioSource reverbSource{&high, false};
    juce::Reverb::Parameters reverbParameters;

    double currentSampleRate = 44100.0;
};
```

```
void DJAudioPlayer::prepareToPlay(int samplesPerBlockExpected, double sampleRate)
{
    currentSampleRate = sampleRate;

    transportSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
    resampleSource.prepareToPlay(samplesPerBlockExpected, sampleRate);

    low.setCoefficients(juce::IIRCoefficients::makeLowShelf(sampleRate, 200.0, 1.0, 1.0f));
    low.prepareToPlay(samplesPerBlockExpected, sampleRate);

    mid.setCoefficients(juce::IIRCoefficients::makePeakFilter(sampleRate, 2000.0, 1.0, 1.0f));
    mid.prepareToPlay(samplesPerBlockExpected, sampleRate);

    high.setCoefficients(juce::IIRCoefficients::makeHighShelf(sampleRate, 8000.0, 1.0, 1.0f));
    high.prepareToPlay(samplesPerBlockExpected, sampleRate);

    reverbSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
}
```

Users use the volume slider to adjust the gain value for the deck. The program takes the slider value and applies it to the audio before parsing it through the rest of the processing chain using AudioTransportSource.

```
/* slider event listener */
void DeckGUI::sliderValueChanged(juce::Slider *slider)
{
    if (slider == &volSlider)
    {
        player->setGain(slider->getValue());
        volValue.setText(juce::String(static_cast<int>(slider->getValue() * 100)) + "%",
            juce::dontSendNotification);
    }
```

```
void DJAudioPlayer::setGain(double gain)
{
    if (gain < 0 || gain > 1.0)
    {
        std::cout << "DJAudioPlayer::setGain gain should be between 0 and 1" << std::endl;
    }
    else
    {
        transportSource.setGain(gain);
    }
}
```

Likewise, the program takes the value from the speed slider and resamples the audio file in real time using the ResamplingAudioSource.setResamplingRatio() method. This changes the speed and pitch of the audio playback while maintaining audio quality.

```
335     if (slider == &speedSlider)
336     {
337         player->setSpeed(slider->getValue());
338         speedValue.setText(juce::String(slider->getValue(), 2) + "x", juce::dontSendNotification);
339     }
```

```
85  void DJAudioPlayer::setSpeed(double ratio)
86  {
87      if (ratio < 0 || ratio > 100.0)
88      {
89          std::cout << "DJAudioPlayer::setSpeed ratio should be between 0 and 1" << std::endl;
90      }
91      else
92      {
93          resampleSource.setResamplingRatio(ratio);
94      }
95  }
```

These elements make up one GUI deck. Each deck is declared in the private section within MainComponent.h, then duplicated in MainComponent::resized().

```
20  private:
21      juce::AudioFormatManager formatManager;
22      juce::AudioThumbnailCache thumbCache{100};
23
24      DJAudioPlayer player1{formatManager};
25      DeckGUI deckGUI1{&player1, formatManager, thumbCache};
26
27      DJAudioPlayer player2{formatManager};
28      DeckGUI deckGUI2{&player2, formatManager, thumbCache};
29
30      juce::MixerAudioSource mixerSource;
31
32      JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(MainComponent)
33  };
```

```
58  void MainComponent::resized()
59  {
60      deckGUI1.setBounds(0, 0, getWidth() / 2, getHeight());
61      deckGUI2.setBounds(getWidth() / 2, 0, getWidth() / 2, getHeight());
62  }
63
```

User interface



Above is a screenshot image of my GUI layout.

There are two identical DJ decks that allow for two different audio tracks to be played and manipulated from. On the top are buttons users can select to import and play audio tracks. The waveform is also displayed at the top of the interface.

In the middle are linear sliders and rotary dials. The largest dial is the position slider that updates its position during audio playback, turning from the minimum value to the maximum value. To the left are the volume controls. The linear slider controls the general loudness of the music, while the dials control the high, mid, and low frequencies. On the right is the speed slider that allows users to adjust playback speed from 0.5 to 2.0 times.

Lastly are the rotary dials that control audio reverb. There are six dials representing each of the six parameters: wet level, dry level, damping, freeze, room size, and width. Below each dial, users can see the values they are adjusting each dial to.

These sliders and dials take user actions as event listeners and adjust values in the program respectively. This allows the program to manipulate the audio file in real-time, enabling a DJ experience for users.

New feature: equalizers

Across many DJ applications I have come across during research, I noticed that equalizers are a staple in DJ software.



Above is a screenshot of the VirtualDJ application. Equalizers can be seen in the middle of the screen. Below is transitions.dj, an online DJ application. Likewise, equalizers are seen in the middle of the screen. It is observed that each deck has its own equalizer controls.



I decided to implement these equalizers as part of the main deck, DeckGUI.cpp. First, the sliders are declared in the private section of the DeckGUI header file. Then, they are made visible, set ranges and values, and given listener events within the DeckGUI.cpp file.

To change the high, mid, and low values, the IIRFilterAudioSource, Reverb and ReverbAudioSource class is used. This is setup in the private section in the DJAudioPlayer.h header file. The sample rate is also initialized as 44.1kHz as it is the standard sampling rate of music.

```
38  private:
39      juce::AudioFormatManager &formatManager;
40      std::unique_ptr<juce::AudioFormatReaderSource> readerSource;
41      juce::AudioTransportSource transportSource;
42      juce::ResamplingAudioSource resampleSource{&transportSource, false, 2};
43
44      juce::IIRFilterAudioSource low{&resampleSource, false};
45      juce::IIRFilterAudioSource mid{&low, false};
46      juce::IIRFilterAudioSource high{&mid, false};
47
48      juce::ReverbAudioSource reverbSource{&high, false};
49      juce::Reverb::Parameters reverbParameters;
50
51      double currentSampleRate = 44100.0;
52  };
53
```

Within DJAudioPlayer::prepareToPlay in DJAudioPlayer.cpp, the equalizer filters are set up to a neutral state using the setCoefficients() method. IIRCoefficients and suitable static functions are used: makeLowShelf, makeHighShelf, and makePeakFilter, to calculate low, mid, and high, values respectively.

It uses the currentSampleRate of 44100.0 reinitialized as sampleRate, frequency value, double Q, and a default gain value. After, the prepareToPlay() method is used to prepare each frequency for playing in the next audio block.

```
18  void DJAudioPlayer::prepareToPlay(int samplesPerBlockExpected, double sampleRate)
19  {
20      currentSampleRate = sampleRate;
21
22      transportSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
23      resampleSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
24
25      low.setCoefficients(juce::IIRCoefficients::makeLowShelf(sampleRate, 200.0, 1.0, 1.0f));
26      low.prepareToPlay(samplesPerBlockExpected, sampleRate);
27
28      mid.setCoefficients(juce::IIRCoefficients::makePeakFilter(sampleRate, 2000.0, 1.0, 1.0f));
29      mid.prepareToPlay(samplesPerBlockExpected, sampleRate);
30
31      high.setCoefficients(juce::IIRCoefficients::makeHighShelf(sampleRate, 8000.0, 1.0, 1.0f));
32      high.prepareToPlay(samplesPerBlockExpected, sampleRate);
33
34      reverbSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
35  }
```

This is called again in DJAudioPlayer::setHigh, DJAudioPlayer::setMid, and DJAudioPlayer::setLow. This allows the program to respond real-time as users adjust the sliders.

```
115 void DJAudioPlayer::setHigh(double highLevel)
116 {
117     if (highLevel < 2.0 && highLevel > 0.0)
118     {
119         high.setCoefficients(juce::IIRCoefficients::makeHighShelf(currentSampleRate, 8000.0, 1.0, static_cast<float>(highLevel)));
120     }
121 }
122
123 void DJAudioPlayer::setMid(double midLevel)
124 {
125     if (midLevel < 2.0 && midLevel > 0.0)
126     {
127         mid.setCoefficients(juce::IIRCoefficients::makePeakFilter(currentSampleRate, 2000.0, 1.0, static_cast<float>(midLevel)));
128     }
129 }
130
131 void DJAudioPlayer::setLow(double lowLevel)
132 {
133     if (lowLevel < 2.0 && lowLevel > 0.0)
134     {
135         low.setCoefficients(juce::IIRCoefficients::makeLowShelf(currentSampleRate, 200.0, 1.0, static_cast<float>(lowLevel)));
136     }
137 }
```

Finally, in releaseResources(), the equalizer values are called again to allow the program to exit gracefully.

```
47 void DJAudioPlayer::releaseResources()
48 {
49     transportSource.releaseResources();
50     resampleSource.releaseResources();
51     reverbSource.releaseResources();
52
53     low.releaseResources();
54     mid.releaseResources();
55     high.releaseResources();
56 }
```

Lastly, a LookAndFeel method is used to create a better looking GUI for the equalizer sliders. Instead of using a standard linear slider, I opted for a rotary slider. This can be set using the setSliderStyle() method.

```
160     /* set slider styles */
161     volSlider.setSliderStyle(juce::Slider::SliderStyle::LinearBarVertical);
162     highSlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
163     midSlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
164     lowSlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
165     posSlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
166     speedSlider.setSliderStyle(juce::Slider::SliderStyle::LinearBarVertical);
167     wetSlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
168     dampingSlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
169     roomSizeSlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
170     drySlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
171     freezeSlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
172     widthSlider.setSliderStyle(juce::Slider::SliderStyle::Rotary);
```

Next, in the LookAndFeel header file, a public method, drawRotarySlider is declared.

```
class rotaryLAF : public juce::LookAndFeel_V4
{
public:
    rotaryLAF();
    ~rotaryLAF();

    void drawRotarySlider(juce::Graphics &g,
                          int x, int y,
                          int width, int height,
                          float sliderPos,
                          float rotaryStartAngle, float rotaryEndAngle,
                          juce::Slider &slider) override;
};
```

In LookAndFeel.cpp, graphics for the rotary slider are created. It is made following a tutorial created by The Audio Programmer (link).

First, the dimensions of the knob are initialized. After, the main body of the rotary slider is drawn and filled with a colour gradient. To draw the position indicator, a small light grey circle is created. This circle rotates about the center of the dial as the user adjusts the slider.

Together, these elements draw a realistic-looking knob that resembles dials on a professional DJ deck.

```
void rotaryLAF::drawRotarySlider(juce::Graphics &g,
                                 int x, int y,
                                 int width, int height,
                                 float sliderPos,
                                 float rotaryStartAngle, float rotaryEndAngle,
                                 juce::Slider &slider)
{
    float diameter = fmin(width, height);
    float radius = diameter / 2;
    float centreX = x + width / 2;
    float centreY = y + height / 2;
    float rx = centreX - radius;
    float ry = centreY - radius;
    auto angle = rotaryStartAngle + (sliderPos * (rotaryEndAngle - rotaryStartAngle));

    // dial main body
    juce::ColourGradient gradient(juce::Colour(40, 40, 40), centreX, centreY,
                                  juce::Colour(25, 25, 25), centreX + radius, centreY + radius,
                                  false);
    g.setGradientFill(gradient);
    g.fillEllipse(rx, ry, diameter, diameter);
    juce::ColourGradient gradient2(juce::Colour(50, 50, 50), centreX, centreY,
                                   juce::Colour(40, 40, 40), centreX + radius, centreY + radius,
                                   false);
    g.setGradientFill(gradient2);
    g.fillEllipse(rx + radius / 2, ry + radius / 2, radius, radius);

    // dial tick
    juce::Path dialTick;
    float tickThickness = 3.0f;
    float innerOffset = 5.0f;
    dialTick.addEllipse(-tickThickness / 2, -radius + innerOffset, tickThickness, tickThickness);
    g.setColour(juce::Colour(211, 211, 211));
    g.fillPath(dialTick, juce::AffineTransform::rotation(angle).translated(centreX, centreY));
};
```

Finally, this LookAndFeel method is applied to the high, mid, and low sliders in DeckGUI.cpp.

```
130 void DeckGUI::appearance()
131 {
132     /* look and feel */
133     volSlider.setLookAndFeel(&linearSlider);
134     speedSlider.setLookAndFeel(&linearSlider);
135     posSlider.setLookAndFeel(&posLAF);
136     wetSlider.setLookAndFeel(&rotaryLAF);
137     dampingSlider.setLookAndFeel(&rotaryLAF);
138     roomSizeSlider.setLookAndFeel(&rotaryLAF);
139     drySlider.setLookAndFeel(&rotaryLAF);
140     freezeSlider.setLookAndFeel(&rotaryLAF);
141     widthSlider.setLookAndFeel(&rotaryLAF);
142     highSlider.setLookAndFeel(&rotaryLAF);
143     midSlider.setLookAndFeel(&rotaryLAF);
144     lowSlider.setLookAndFeel(&rotaryLAF);
145
```

This gives us the final product as seen below. The high, mid, and low values manipulates the frequency levels of audio files loaded into the program.



New feature: reverb

One of the popular features on DJ decks is the reverb effect. In applications like Virtual DJ, users are able to adjust various parameters within the reverb scope. In the case of Virtual DJ, it has parameters such as strength, room size, diffusion, and brightness.



(via HOT TECH UG: link)

To create the reverb effect in my DJ application, I used the JUCE reverb class. Its parameters are room size, damping, wet level, dry level, width, and freeze mode. These parameters can be manipulated using rotary dials similar to the one used for equalizers.

The reverb dials are implemented on the main deck in DeckGUI.cpp. Similar to the previous extension, the sliders for the individual parameters are declared in the private section of the DeckGUI header file. They are made visible, given ranges and initial values, and given listener events within the DeckGUI.cpp file.

```
38  private:
39      juce::AudioFormatManager &formatManager;
40      std::unique_ptr<juce::AudioFormatReaderSource> readerSource;
41      juce::AudioTransportSource transportSource;
42      juce::ResamplingAudioSource resampleSource{&transportSource, false, 2};
43
44      juce::IIRFilterAudioSource low{&resampleSource, false};
45      juce::IIRFilterAudioSource mid{&low, false};
46      juce::IIRFilterAudioSource high{&mid, false};
47
48      juce::ReverbAudioSource reverbSource{&high, false};
49      juce::Reverb::Parameters reverbParameters;
50
51      double currentSampleRate = 44100.0;
52  };
53
```

ReverbAudioClass class, renamed reverbSource, is set up in DJAudioPlayer.h alongside the parameters function. In DJAudioPlayer::prepareToPlay in DJAudioPlayer.cpp, reverbSource is called. It is also called in DJAudioPlayer::getNextAudioBlock.

```
18  void DJAudioPlayer::prepareToPlay(int samplesPerBlockExpected, double sampleRate)
19  {
20      currentSampleRate = sampleRate;
21
22      transportSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
23      resampleSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
24
25      low.setCoefficients(juce::IIRCoefficients::makeLowShelf(sampleRate, 200.0, 1.0, 1.0f));
26      low.prepareToPlay(samplesPerBlockExpected, sampleRate);
27
28      mid.setCoefficients(juce::IIRCoefficients::makePeakFilter(sampleRate, 2000.0, 1.0, 1.0f));
29      mid.prepareToPlay(samplesPerBlockExpected, sampleRate);
30
31      high.setCoefficients(juce::IIRCoefficients::makeHighShelf(sampleRate, 8000.0, 1.0, 1.0f));
32      high.prepareToPlay(samplesPerBlockExpected, sampleRate);
33
34      reverbSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
35  }
```

```
37  void DJAudioPlayer::getNextAudioBlock(const juce::AudioSourceChannelInfo &bufferToFill)
38  {
39      if (readerSource.get() == nullptr)
40      {
41          bufferToFill.buffer->clear(bufferToFill.startSample, bufferToFill.numSamples);
42          return;
43      }
44      reverbSource.getNextAudioBlock(bufferToFill);
45  }
```

As users adjust the rotary dials, the program takes the dial value and assigns it to the reverb parameter. reverbSource.setParameters(reverbParameters) applies the changes.

```
139 void DJAudioPlayer::setWet(float wet)
140 {
141     if (wet < 0 || wet > 1.0)
142     {
143         std::cout << "DJAudioPlayer::setWet wet level should be between 0 and 1" << std::endl;
144     }
145     else
146     {
147         reverbParameters.wetLevel = wet;
148         reverbSource.setParameters(reverbParameters);
149     }
150 }
151
152 void DJAudioPlayer::setDamping(float damping)
153 {
154     if (damping < 0 || damping > 1.0)
155     {
156         std::cout << "DJAudioPlayer::setDamping damping level should be between 0 and 1" << std::endl;
157     }
158     else
159     {
160         reverbParameters.damping = damping;
161         reverbSource.setParameters(reverbParameters);
162     }
163 }
```

```
165 void DJAudioPlayer::setRoomSize(float roomSize)
166 {
167     if (roomSize < 0 || roomSize > 1.0)
168     {
169         std::cout << "DJAudioPlayer::setRoomSize size should be between 0 and 1" << std::endl;
170     }
171     else
172     {
173         reverbParameters.roomSize = roomSize;
174         reverbSource.setParameters(reverbParameters);
175     }
176 }
177
178 void DJAudioPlayer::setDry(float dry)
179 {
180     if (dry < 0 || dry > 1.0)
181     {
182         std::cout << "DJAudioPlayer::setDry dry level should be between 0 and 1" << std::endl;
183     }
184     else
185     {
186         reverbParameters.dryLevel = dry;
187         reverbSource.setParameters(reverbParameters);
188     }
189 }
```

```
191 void DJAudioPlayer::setFreeze(float freeze)
192 {
193     reverbParameters.freezeMode = freeze;
194     reverbSource.setParameters(reverbParameters);
195 }
196
197 void DJAudioPlayer::setWidth(float width)
198 {
199     if (width < 0 || width > 1.0)
200     {
201         std::cout << "DJAudioPlayer::setWidth width should be between 0 and 1" << std::endl;
202     }
203     else
204     {
205         reverbParameters.width = width;
206         reverbSource.setParameters(reverbParameters);
207     }
208 }
```

Finally, in releaseResources(), the reverb parameters are called again for seamless program exit. The reverb dial design uses the same LookAndFeel method from the previous extension, equalizers.

```
47 void DJAudioPlayer::releaseResources()
48 {
49     transportSource.releaseResources();
50     resampleSource.releaseResources();
51     reverbSource.releaseResources();
52
53     low.releaseResources();
54     mid.releaseResources();
55     high.releaseResources();
56 }
```

```
130 void DeckGUI::appearance()
131 {
132     /* look and feel */
133     volSlider.setLookAndFeel(&linearSlider);
134     speedSlider.setLookAndFeel(&linearSlider);
135     posSlider.setLookAndFeel(&posLAF);
136     wetSlider.setLookAndFeel(&rotaryLAF);
137     dampingSlider.setLookAndFeel(&rotaryLAF);
138     roomSizeSlider.setLookAndFeel(&rotaryLAF);
139     drySlider.setLookAndFeel(&rotaryLAF);
140     freezeSlider.setLookAndFeel(&rotaryLAF);
141     widthSlider.setLookAndFeel(&rotaryLAF);
142     highSlider.setLookAndFeel(&rotaryLAF);
143     midSlider.setLookAndFeel(&rotaryLAF);
144     lowSlider.setLookAndFeel(&rotaryLAF);
```